

Rust

Page No.:

Date: / /

```
# let least = std::cmp::min(3, 8);  
           ↓      ↓      ↓ (approximately)  
           crate::file::function  
           (lib) (module)
```

```
use std::cmp::min;  
let least = min(7, 1); → 1
```

```
let x = "amos".len();  
let x = str::len("amos"); → 4
```

Structs are declared with struct keyword

```
struct Number {  
    odd: bool,  
    value: i32,  
}
```

→ They can be initialised using literals.

```
let x = Number { odd: false, value: 2 };  
let y = Number { value: 3, odd: true };
```

```
# fn print_number(n: Number) {  
    match n.value {  
        1 => println!("One"),  
        2 => println!("Two"),  
        _ => println!("??", n.value),  
    }  
}  
  
impl Number {  
    fn is_positive(self) -> bool {  
        self.value > 0  
    }  
}
```

→ to avoid method taking ownership of self

Functions can be generic:

```
fn fooBar<T>(arg: T) {
    // work with 'arg'
}
```

```
fn fooBar<L, R>(left: L, right: R) {
    // work with 'left' & 'right'
}
```

Structs can be generic too:

```
struct Pair<T> {
    a: T,
    b: T,
}
```

```
let p1 = Pair { a: 3, b: 9 };
// = Pair<Int>
```

```
let p2 = Pair { a: true, b: false };
// = Pair<Bool>
```

```
let mut v1 = Vec::new();
v1.push(1);
```

```
let mut v2 = Vec::new();
// v1 == Vec<Int>
// v2 == Vec<Bool>
```

```
fn main() {
```

```
    let v1 = vec![1, 2, 3];
```

```
    let v2 = vec![true, false, true];
```

```
}
```


Iterators

let natural-numbers = 1..∞;
(all natural no from 1 to ∞)

// 0 or greater

(0..) .contains(100); // true

// 20 or less than 20

(..=20) .contains(20); // true

// only 3, 4, 5

(3..6) .contains(4); // true

```
fn main() {  
  for c in "rust".chars() {  
    println!("Give me a {}", c);  
  }  
}
```

→ Give me a r/u/s/t

```
fn main() {  
  for c in "SURPRISE INBOUND".chars() {  
    .filter(|c| c.is_lowercase())  
    .flat_map(|c| c.to_uppercase())  
  }  
  print!("{}", c);  
}
```

// output : UB

* let mut $i = N$;

for i in $1..$, take-while($\{ \&n \mid n \leq i \}$)

{ $i = i/3$ }

* To take input variable in rust

use std::io;

• Create a mutable string variable to store input

let mut input = String::new();

• Read the user input

io::stdin().read-line(&mut input)

• expect("failed to read line");

• Print input back to user.

println!("Hello, {}!", input);


```
fn main() {
```

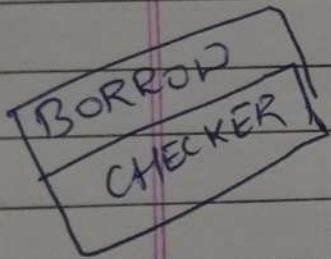
```
  let mut hello: Vec<i32> = (0..10).collect();
```

mutable name type value

```
  fn do_stuff (val: ← Borrowing &Vec<i32>) {
    println!("{}", val.len()); } MACRO!
```

```
  }
```

```
  do_stuff(&hello);
```



• unwrap(); → I am ok with runtime exception